

Practical Session 14: Summarizing & Extracting from SEC Filings

Hands-On — Extractive Summarization, ROUGE, and Consistency Checking

Juan F. Imbet

Paris Dauphine – PSL University

June 20, 2026

Duration: 1 hour (50 min problems + 10 min debrief)

Timed agenda

- ➊ **Recap** (5 min): extractive vs. abstractive; ROUGE- N ; consistency.
- ➋ **Problem 1** (12 min): compute ROUGE-1/2/L by hand for two candidate summaries.
- ➌ **Problem 2** (13 min): design a faithfulness-checking pipeline; classify three summary claims as verified / unverifiable / incorrect.
- ➍ **Case Study** (20 min): run the Python 10-K pipeline — section routing, TextRank extractive summary, ROUGE, number consistency.
- ➎ **Discussion** (10 min): extractive vs. abstractive in production.

Code lives in `code/notebooks/14-financial-text-summarization/`.

Recap: Formulas You Will Need

ROUGE- N (recall-oriented):

$$\frac{\#\{\text{matched } n\text{-grams}\}}{\#\{n\text{-grams in reference}\}}$$

ROUGE-L: LCS-based; uses the longest common subsequence between candidate and reference.

Extractive vs. abstractive:

- Extractive = verbatim sentences → faithfulness free.
- Abstractive = generated → fluent but hallucination-prone.

Consistency-check recipe (3 steps)

(1) Multi-source extraction of each metric → (2) normalise to common unit/sign → (3) cross-reference; flag pairs disagreeing beyond tolerance (e.g. 0.1%).

Faithfulness label set: *verified* (matches source) / *unverifiable* (not in source) / *incorrect* (contradicts source).

Problem 1: ROUGE by Hand

Reference summary R :

"revenue rose 13 percent to 56.5 billion"

Candidate A: "revenue rose 13 percent"

Candidate B: "profit fell 13 percent to 56.5 billion"

Tasks (treat each word as a token; lowercase):

- A. Compute **ROUGE-1** (unigram recall) for A and for B.
- B. Compute **ROUGE-2** (bigram recall) for A and for B.
- C. Compute **ROUGE-L** (LCS recall) for A and for B.

Discussion question

Candidate B paraphrases "revenue rose" as the *semantically opposite* "profit fell" yet shares many tokens. What does its score reveal about ROUGE's core weakness, and which metric from the lecture would catch the error?

Problem 2: Designing a Faithfulness Check

Source facts (from the filing / XBRL): revenue = \$56.5B (+13% YoY); net income = \$16.4B; Azure growth = +29%.

Candidate abstractive summary:

“Revenue grew 13% to \$56.5 billion. Net income rose to \$17 billion. Azure cloud revenue increased 29%, and management raised full-year guidance.”

Tasks:

- A. Label each of the four claims: verified / unverifiable / incorrect.
- B. For the numerical claims, describe how to ground them against the *extracted* XBRL data (which step of the 3-step consistency recipe?).
- C. Which claim is the most dangerous in a decision context, and why?

Case Study: An End-to-End 10-K Mini-Pipeline

Goal: take a 10-K excerpt and run the full chapter pipeline — section routing → extractive summary → ROUGE vs. a reference → number-consistency check.

Stack (pure Python, no API key needed):

- `re` — section routing & monetary-figure extraction
- `networkx` — TextRank graph (sentence similarity → PageRank)
- `rouge_score` (or the hand formula) — evaluation

What you will do:

- 1 Run the provided code on the sample MD&A text.
- 2 Inspect which sentences TextRank selects.
- 3 Run the consistency check; see which prose number disagrees with the table.

Setup

`pip install networkx rouge-score` — no model download or API key required; everything runs locally in seconds.

Case Study: The Code (Part 1a — Setup & Helpers)

```
import re, networkx as nx
from collections import Counter

MDNA = (
    "Revenue increased 13% to $56.5 billion in fiscal 2023. "
    "The growth was driven by strong cloud demand. "
    "Azure revenue grew 29% year over year. "
    "Operating margin expanded to 42%. "
    "Management expects continued momentum into fiscal 2024. "
    "The company faces foreign-exchange headwinds."
)

def sentences(text):
    return [s.strip() for s in re.split(r'(?<=[.!?])\s+', text) if s.strip()]

def bow(s):
    # bag-of-words vector
    return Counter(re.findall(r"[a-z]+", s.lower()))

def sim(a, b):
    # cosine over word counts
    common = set(a) & set(b)
    num = sum(a[w] * b[w] for w in common)
    den = (sum(v*v for v in a.values())**0.5) * \
        (sum(v*v for v in b.values())**0.5)
    return num / den if den else 0.0
```

Case Study: The Code (Part 1b — TextRank)

```
def textrank(text, k=2):
    sents = sentences(text); vecs = [bow(s) for s in sents]
    G = nx.Graph()
    for i in range(len(sents)):
        for j in range(i+1, len(sents)):
            w = sim(vecs[i], vecs[j])
            if w > 0: G.add_edge(i, j, weight=w)
    scores = nx.pagerank(G)
    top = sorted(scores, key=scores.get, reverse=True)[:k]
    return [sents[i] for i in sorted(top)]

print(textrank(MDNA, k=2))
```

Idea: sentences vote for each other via cosine similarity; PageRank surfaces the most central (salient) ones — selected *verbatim*, so the extractive summary is faithful by construction.

Case Study: The Code (Part 2 — Consistency Check)

```
# Table (XBRL-derived) ground truth, in USD millions
TABLE = {"revenue": 56500.0, "azure_growth_pct": 29.0}

# Numbers stated in the prose narrative
PROSE = {"revenue": 56500.0, "azure_growth_pct": 28.0} # note: 28 vs 29

def consistency(table, prose, tol=0.001):
    flags = []
    for k in table.keys() & prose.keys():
        t, p = table[k], prose[k]
        rel = abs(t - p) / abs(t) if t else 0.0
        status = "OK" if rel <= tol else "MISMATCH"
        flags.append((k, t, p, f"{rel:.2%}", status))
    return flags

for row in consistency(TABLE, PROSE):
    print(row)

# -> ('revenue', 56500.0, 56500.0, '0.00%', 'OK')
# -> ('azure_growth_pct', 29.0, 28.0, '3.45%', 'MISMATCH')
```

Case Study: Diagnostic Questions

After running both code parts, work through the following:

- 1 Which sentences did TextRank select? Do they cover revenue *and* guidance?
- 2 The consistency check flags `azure_growth_pct`. Is this an *extraction error* or a *disclosure inconsistency*? How would you tell?
- 3 Set `tol = 0.05`. Does the mismatch still flag? What is the danger of a loose tolerance?

Discussion: Extractive vs. Abstractive in Production

For each scenario, choose extractive, abstractive, or hybrid — and justify.

- 1 A **covenant-monitoring** system summarises loan-agreement clauses for a credit committee. *Every numerical threshold must be exact.*
- 2 An **equity desk** wants a fluent 150-word daily digest of overnight analyst notes for human readers who will not act on it directly.
- 3 A **regulator-facing** 10-K summary that must be auditable: every claim traceable to a source sentence.

Frame your answer around

Faithfulness vs. fluency · hallucination cost · auditability / traceability · whether a human or a downstream system consumes the output · availability of a structured (XBRL) ground truth for grounding.

Solution: Problem 1 — ROUGE by Hand

Reference R (7 tokens): revenue rose 13 percent to 56.5 billion.

Candidate A = revenue rose 13 percent:

- ROUGE-1 = $4/7 \approx 0.571$ (revenue, rose, 13, percent matched).
- ROUGE-2 = $3/6 = 0.500$ (bigrams “revenue rose”, “rose 13”, “13 percent”).
- ROUGE-L = LCS length $4/7 \approx 0.571$ (the 4 words appear in order).

Candidate B = profit fell 13 percent to 56.5 billion:

- ROUGE-1 = $5/7 \approx 0.714$ (13, percent, to, 56.5, billion matched).
- ROUGE-2 = $4/6 \approx 0.667$ (“13 percent”, “percent to”, “to 56.5”, “56.5 billion”).
- ROUGE-L = LCS = $5/7 \approx 0.714$.

Solution: Problem 1 — The Lesson

Recall the scores: Candidate A (revenue rose 13 percent) scored ≈ 0.571 ; Candidate B (profit fell 13 percent to 56.5 billion) scored ≈ 0.714 on every ROUGE variant.

The lesson

B *outscores* A on every ROUGE variant despite **inverting the meaning** (“revenue rose” → “profit fell”). ROUGE rewards lexical overlap, not semantics or factual direction — exactly why you need **factual-consistency / NLI-based** checks (FactCC) for finance.

Solution: Problem 2 — Faithfulness Check

A. Claim labels:

- “Revenue grew 13% to \$56.5B” → **verified** (matches source).
- “Net income rose to \$17 billion” → **incorrect** (source: \$16.4B; rounding to \$17B is a *material* misstatement).
- “Azure increased 29%” → **verified**.
- “Management raised full-year guidance” → **unverifiable** (no guidance fact in the provided source).

B. Grounding numbers against XBRL: this is step (3), *cross-reference* — after multi-source extraction and normalisation, compare each prose number to the XBRL figure; \$17B vs. \$16.4B exceeds a 0.1% tolerance ($\sim 3.7\%$) → flag.

C. Most dangerous: the *net income* claim — it is confidently stated, numerically wrong, and feeds directly into valuation/EPS reasoning. The “unverifiable” guidance claim is also risky (potential extrinsic hallucination) but is detectable as ungrounded rather than contradicted.

Wrap-Up: Key Takeaways

- **ROUGE is lexical**: high overlap can coexist with inverted meaning. Always pair with semantic (BERTScore) *and* factual (NLI / XBRL-grounded) checks.
- **Extractive summarization** buys faithfulness for free — TextRank selects salient verbatim sentences via sentence-graph PageRank.
- **Consistency checking** is a cheap, high-value quality gate: normalise, then cross-reference prose numbers against the XBRL table.
- **Tolerance matters**: too loose a threshold hides material errors; too tight flags rounding noise.
- **Choose the paradigm by the stakes**: exact numbers / auditability → extractive; fluent human digests → abstractive (with checks).

Next practical

Fine-tuning SEC-BERT for FINER-139 NER and benchmarking against a zero-shot LLM extractor on earnings-call entity recognition.

APPENDIX

Stretch Problems

Extra / optional material — beyond the core lecture.

Appendix: Stretch Problem — Hierarchical S-1

An S-1 runs 300,000 words — beyond any single context window.

Task: sketch a two-stage hierarchical summarizer.

- 1 Stage 1: each of ~ 30 sections $\rightarrow$ 200–400-word summary. What is the approximate total intermediate token budget?
- 2 Stage 2: concatenate section summaries; second-pass model $\rightarrow$ final summary.
- 3 Compare cost vs. a retrieval-augmented approach that only summarises the top- k passages for “principal risk factors”. When does each win?

Appendix: Stretch Problem — Arithmetic via Code

The lecture's *ReAct* principle: route arithmetic to a code interpreter.

Task: given extracted `revenue_2023 = 56500` and `revenue_2022 = 50000` (USD M), an LLM is asked for YoY growth.

- 1 Why might the LLM answer “~11%” or “15%” when computing inline?
- 2 Write the one-line Python the agent should emit instead: $(56500 - 50000) / 50000 * 100$.
- 3 State the verified result and explain how the “decide-what-to-compute” vs. “actually-compute” separation eliminates the numerical error class.